

LAMMPS Lennard-Jones Melt Report

HPC-I Chap. 5-2 Packages Assignment

112006265,
TING-YU TSAI,
EECS (GS)

Testcase	Lennard-Jones Melt (lj-melt)
Build script submitted	build_all.sh
Build method	Source build using CMake with MPI enabled
Comparison	Baseline build vs. OpenMP/compiler-flag build
Main finding	The OpenMP/compiler-flag build was slower in the actual timing logs

1. Testcase and Goal

I chose the Lennard-Jones Melt testcase. My goal was to make one reasonable build comparison for the assignment: submit a working build script, explain the compile options, and show that the simulation produced usable log and visualization output. I am confident that I completed the workflow and read the logs correctly, but I am not claiming that this is a perfect performance study.

The important correction in this version is the runtime result. My earlier summary treated the OpenMP/compiler-flag build as faster, but the actual log files show the opposite. The baseline run was faster for this test. I think this is still a valid result because the assignment asks for optimization analysis, not only a successful speedup.

2. Installation / Build Script Used

I used build_all.sh as the submitted installation script. The script clones the official LAMMPS stable branch if the lammps folder does not already exist, then builds two executables: Imp_baseline and Imp_optimized. I do not paste the full script here because it is submitted separately, but the key settings are summarized below.

Build	Main CMake / compiler settings	Output binary
Baseline	Release; CMAKE_CXX_FLAGS="-O2"; BUILD_MPI=yes; BUILD_OMP=no; PKG_OPENMP=no	Imp_baseline
OpenMP/compiler-flag build	Release; CMAKE_CXX_FLAGS="-O3 - march=native -ffast-math"; BUILD_MPI=yes; BUILD_OMP=yes; PKG_OPENMP=yes	Imp_optimized

3. Compile Option Insight

The baseline build uses -O2 and disables the LAMMPS OpenMP package. This gives me a simple reference point.

The second build uses -O3, -march=native, -ffast-math, and enables BUILD_OMP / PKG_OPENMP. My expectation was that the Lennard-Jones pair-force work might benefit because lj-melt repeatedly computes short-range interactions over many neighboring atoms. The actual timing logs show that enabling OpenMP in this configuration did not automatically improve the run.

-O3 and -march=native can help the compiler generate more optimized CPU code. -ffast-math may also allow faster floating-point transformations, but it can slightly change floating-point behavior.

LAMMPS Lennard-Jones Melt Report

HPC-I Chap. 5-2 Packages Assignment

112006265,
TING-YU TSAI,
EECS (GS)

4. Runtime Log Summary

The timing logs showed that the OpenMP/compiler-flag build was slower for this lj-melt run. The baseline used 4 MPI tasks with no OpenMP threads, while the second run used 4 MPI tasks with 2 OpenMP threads per MPI task. The main timing values are below.

Timing result	Baseline	OpenMP/compiler-flag build	Observation
Total wall time	0:13:08	0:47:38	second run was slower
Final loop time	232.304 s	893.627 s	slower final stage
Timesteps/s	64.571	16.786	lower throughput
Parallel setup	4 MPI tasks, no OpenMP	4 MPI tasks x 2 OpenMP threads	different execution mode
Dangerous builds	0	0	both runs completed stably

I successfully built and tested two configurations, but the OpenMP/compiler-flag configuration was not faster for this specific run.

One likely reason is overhead. OpenMP can help when the threaded work is large enough, but it also adds thread-management and synchronization cost. In this run, the Modify section became very large in the OpenMP run, and communication/other overheads were also more visible. The pair-force percentage became smaller, but the total time was still worse, so percentage alone would be misleading.

I am confident about the log reading, but not 100% certain about the root cause. To separate the causes properly, I would need extra ablation runs: for example, -O2 vs. -O3 first, then OpenMP on/off separately, and then different MPI/OpenMP layouts such as 8 MPI tasks x 1 thread or 2 MPI tasks x 4 threads.

5. Logs and Visualization Results

Even though the OpenMP/compiler-flag build was slower, the simulation itself still produced reasonable output. I plotted the thermodynamic log with Python. The temperature rises to about $T^* = 3.0$, stays near that value, and then cools down. Pair energy and pressure generally increase during heating and decrease during cooling, which matches the intended heating-hold-cooling schedule.

LAMMPS Lennard-Jones Melt Report

HPC-I Chap. 5-2 Packages Assignment

112006265,
TING-YU TSAI,
EECS (GS)

Thermodynamic Profile of Lennard-Jones Melting & Cooling Cycle

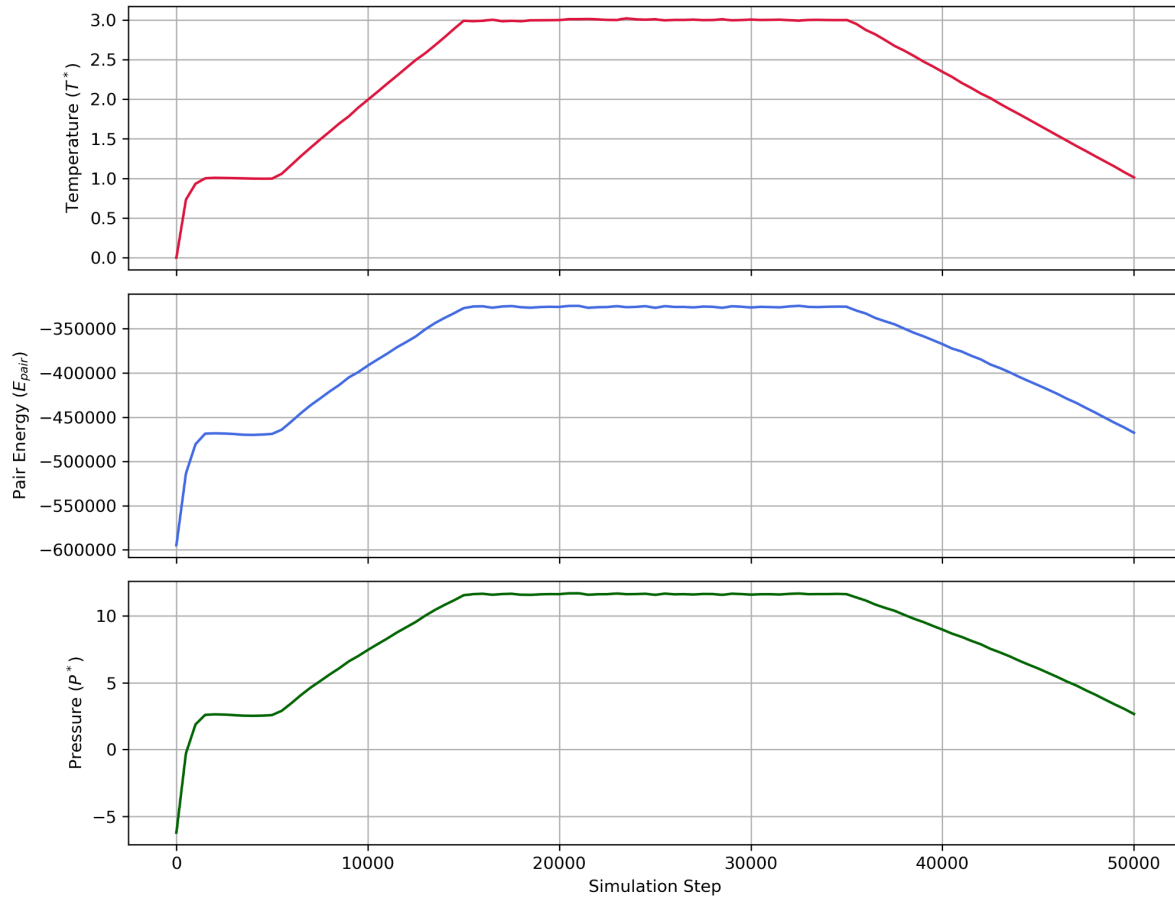


Figure 1. Thermodynamic profile from the Lennard-Jones melt/cooling run.

LAMMPS Lennard-Jones Melt Report

HPC-I Chap. 5-2 Packages Assignment

112006265,
TING-YU TSAI,
EECS (GS)

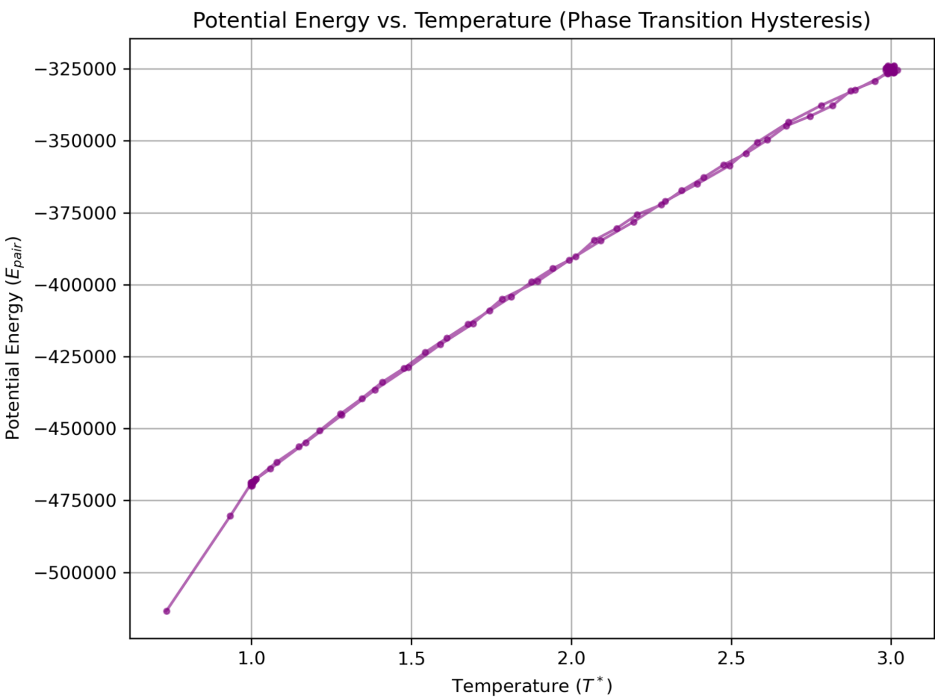
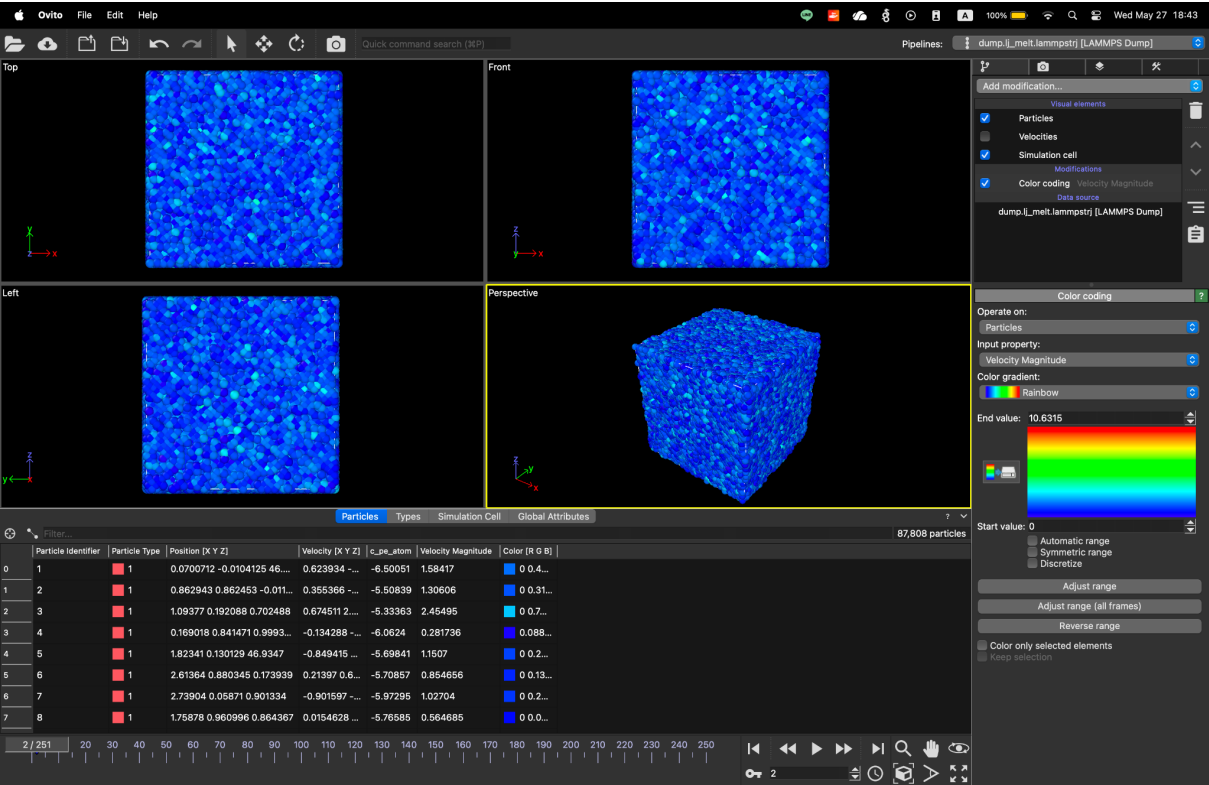


Figure 2. Potential energy versus temperature plot.



LAMMPS Lennard-Jones Melt Report

HPC-I Chap. 5-2 Packages Assignment

112006265,
TING-YU TSAI,
EECS (GS)

Figure 3. OVITO visualization of the generated LAMMPS trajectory.

6. Conclusion

For this assignment, I completed a source-build workflow and compared two LAMMPS configurations for the Lennard-Jones Melt testcase. The baseline binary used -O2 without the OpenMP package, while the second binary used -O3 -march=native -ffast-math with OpenMP support enabled.

The main result is that the OpenMP/compiler-flag build was slower in my actual timing logs. This is not the result I originally expected, but it is still useful: it shows that enabling OpenMP and stronger flags does not automatically produce better performance. For this testcase and runtime layout, the overhead appears to outweigh the benefit.

My confidence level is: I am confident that the build script, runs, logs, plots, and OVITO visualization are valid evidence for the assignment. I am not fully certain about the exact performance cause without more isolated tests. Therefore, I present the result as a careful optimization analysis rather than an overclaimed speedup.

Confidence / Limitation Summary

I am confident about	I am not claiming with certainty
The script built two LAMMPS binaries.	That the second build is generally better.
The baseline run was faster in the actual logs.	The exact individual contribution of OpenMP, -O3, -march=native, or -ffast-math.
The plots and OVITO screenshot show a successful lj-melt run.	That the final structure is perfectly recrystallized without CNA/RDF/defect analysis.